

Binarios SUID y credenciales efectivas

Objetivos de aprendizaje

Modelar UID real, efectivo y guardado; verificar cuándo SUID se aplica o se ignora; reconocer pérdida de privilegios y derivar capacidades del binario.

Prerrequisitos

Credenciales de proceso, permisos, mounts, execve e intérpretes.

Fundamentos técnicos

Al ejecutar un binario SUID, execve puede establecer el UID efectivo al propietario. No cambia automáticamente el UID real. El efecto se ignora con `no_new_privs`, `mount nosuid` o `tracing`, y Linux ignora SUID en scripts. El programa o un intérprete puede bajar privilegios; Bash sin modo privilegiado restablece EUID cuando difiere del real.

Modelo mental

archivo SUID + propietario -> execve -> RUID/EUID/SUID guardado
-> lógica del binario -> posible drop -> operación sensible

Superficie de ataque

Binarios no estándar, helpers propios, versiones concretas y funciones que leen, escriben o ejecutan con EUID superior.

Cómo identificarla

- Propietario y bit con `find/stat`.
- Mount y `no_new_privs` permiten el efecto.
- `id -ru` e `id -u` difieren dentro del proceso.
- Una función alcanzable usa el EUID antes de bajarlo.

Preguntas que debo hacerme

1. ¿Quién es propietario?
2. ¿Binario ELF o script?
3. ¿Mount `nosuid` o `no_new_privs`?
4. ¿El programa baja privilegios?
5. ¿Qué operación concreta ejecuta con EUID elevado?

Prueba mínima

Enumerar y, en un helper del laboratorio, imprimir RUID/EUID y realizar una operación marcador permitida. No asumir que un prompt `#` refleja identidad.

Construcción progresiva del payload

1. `stat` del candidato.
2. Tipo de fichero y mount.
3. RUID/EUID observados.
4. Flujo de función alcanzable.
5. Marcador de lectura/escritura.
6. Control con `no_new_privs` o copia sin bit.

Anatomía de los payloads

- Contexto de entrada: helper SUID root /opt/lab/report-reader.
- Sintaxis original: acepta un ID de reporte, no una ruta.
- Entrada controlada: ID numérico.
- Transformaciones conocidas: conversión decimal y consulta de allowlist.
- Parser final: lógica del helper.
- Sink: lectura de reporte como EUID 0.
- Primitiva: leer solo reporte permitido.
- Payload mínimo: ID de marcador 7.
- Significado de cada componente: 7 selecciona un reporte permitido sin introducir ruta ni metacarácter.
- Resultado esperado: lectura del marcador con EUID 0 conservada en la traza.
- Control negativo: ID inexistente y ruta textual rechazada.
- Restricción observada: allowlist interna.
- Por qué falla la variante básica: pasar /etc/shadow no es ID.
- Hipótesis de adaptación: ninguna sin otra debilidad.
- Payload adaptado: no procede mientras la allowlist y la conversión sean correctas.
- Por qué debería funcionar: no se afirma una adaptación sin una nueva precondition observable.
- Evidencia: EUID 0 y contenido del marcador.
- Cuándo no funcionaría: nosuid, no_new_privs, propietario no root o drop previo.

Variaciones según el contexto

Una función de lectura puede dar impacto sin shell. Una shell hija puede perder EUID. Capabilities y sudo no son equivalentes a SUID y tienen reglas distintas.

Filtros y bypasses

No se “bypassea” nosuid con una receta de otro binario. Si el kernel ignora el bit, la precondition falta. La adaptación debe buscar otra capacidad real, no caracteres.

Evidencias de confirmación

EUID superior dentro de una operación sensible y efecto controlado. Listar un bit solo es hallazgo.

Escalado de impacto

Analizar funciones y versión, demostrar capacidad mínima y evitar invocar shells que alteren credenciales sin comprender su modo privilegiado.

Errores frecuentes

- Decir que SUID “siempre” ejecuta como propietario.
- Tratar scripts SUID como binarios.
- Confundir RUID y EUID.
- Usar /bin/bash sin considerar pérdida de privilegios.
- Ignorar mounts y no_new_privs.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
bit existe, EUID no cambia	nosuid/no_new_privs/tracing	mount y /proc/self/status
shell vuelve al usuario	intérprete baja EUID	comparar helper interno y Bash -p documentado
script SUID no eleva	kernel ignora bit en scripts	file y execve(2)
EUID cambia, lectura falla	permisos/MAC/drop previo	momento exacto y logs
propietario no root	identidad distinta	stat e id

Mitigaciones

Eliminar bits innecesarios, reducir superficie, drop temprano e irreversible, rutas/IDs cerrados, mounts nosuid, no_new_privs, revisión de binarios y actualizaciones.

Relación con pentesting y certificaciones

Se evalúa el modelo de credenciales y la función alcanzable, no ejecutar una línea de catálogo.

Caso guiado

Caso A — Básico: lector limitado

Un helper SUID root imprime RUID 1001/EUID 0 y lee un reporte por ID. Conclusión: SUID activo y capacidad de lectura limitada; no implica shell.

Caso de adaptación

Caso B — Copia en /mnt/share

El mismo binario copiado conserva el bit, pero EUID no cambia. Experimento: findmnt -no OPTIONS /mnt/share. Resultado: nosuid. Conclusión: el payload no falla por versión; falta la precondition del mount.

Caso C — Transferencia: helper de impresión

Un binario SUID de otro usuario escribe colas en su directorio. El alumno demuestra escritura con marcador y evalúa si esa identidad aporta acceso útil. La primitiva es cambio de identidad efectiva, no necesariamente root.

Caso D — Falso positivo: script con bit SUID

deploy.sh muestra -rwsr-xr-x, pero se ejecuta con EUID del alumno. Experimento: file, id dentro y manual execve. Conclusión: Linux ignora el bit en scripts.

Ejercicios

1. Interpreta RUID/EUID/SUID guardado en tres procesos.
2. Distingue nosuid de drop de privilegios.
3. Explica por qué una shell hija puede perder EUID.
4. Evalúa un SUID propiedad de usuario backup.

Resumen

SUID modifica credenciales bajo condiciones concretas. La explotación depende de una función sensible ejecutada antes de perder privilegios.

Chuleta operativa

1. Propietario y tipo.
2. Mount.
3. no_new_privs.
4. RUID/EUID.
5. Drop.
6. Función sensible.
7. Marcador/control.

Referencias

- [execve\(2\)](#)
- [GNU Bash Reference Manual](#)
- [Concepto interno](#) (suid)

Navegación

Anterior: [Sudo](#). Siguiente: [Cron](#). Índice: [curso](#).